
Day 3

List indexing:

Elements of a list can be accessed using their positions, i.e, indexing. Python uses ***zero-based indexing***, meaning the first element is at index '***0***'. It also supports ***negative-indexing*** starting from the end of the list at '***-1***'.

Zero-based indexing:

```
a = [10, 20, 30, 40, 50]

print(a[0])
print(a[2])
```

```
Output

10
30
```

Negative indexing:

```
a = [10, 20, 30, 40, 50]

print(a[-1])
print(a[-3])
```

```
Output

50
30
```

List slicing:

It is a way to access multiple elements from a list. It is used to extract a substring (range of elements).

- The starting index is included, but not the ending index.

```
a = [10, 20, 30, 40, 50]

print(a[1:4])
print(a[:3])
print(a[2:])
```

Output

```
[20, 30, 40]
[10, 20, 30]
[30, 40, 50]
```

List mutability:

List are mutable, meaning its elements can be modified/changed after creating them. Elements can be added, removed or modified without creating a new one.

```
a = [1, 2, 3, 4, 5]

# Modifying an element
a[2] = 10
print(a)
```

Output

```
[1, 2, 10, 4, 5]
```

```python
a = [1, 2, 3, 4, 5]

# Appending 6 to the end of the list
a.append(6)
print(a)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

```python
a = [1, 2, 3, 4, 5]

# Removing element with value 3 from the list
a.remove(3)
print(a)
```

Output

```
[1, 2, 4, 5]
```

```python
a= [1, 2, 3]

# Add elements of another list
a.extend([4, 5, 6])
print(a)
```

Output

```
[1, 2, 3, 4, 5, 6]
```

```
a = [1, 2, 3, 4, 5]

# Slicing and modifying a sublist
a[1:4] = [20, 30, 40]
print(a)
```

Output

```
[1, 20, 30, 40, 5]
```

Nested lists:

It is a list that contains lists as its elements. It is commonly used to represent tabular data.

- Nested elements can be accessed by chaining multiple indexes.

```
a = [[1, 2], [3, 4]]
print(a[0])
print(a[1][0])
```

Output

```
[1, 2]
3
```

Tuple immutability:

Tuple is a collection of objects separated by commas, and they are immutable, meaning elements once created can't be changed or modified.

```
tup = (10, 20, 30)

print(tup)
print(type(tup))
```

Output

```
(10, 20, 30)
<class 'tuple'>
```

They can also be created using the "***tuple()***" operator.

```
numbers = [10, 20, 30]
tup = tuple(numbers)

print(tup)
```

Output

```
(10, 20, 30)
```

Tuple unpacking:

It is a process where individual values can be extracted from a tuple and assign them to multiple variables in a single step.

Key rules:

1. **Number** of variables on the **left** must **match** the **number** of elements in the **tuple**.
2. If they do ***not match***, '***ValueError***' is raised.

```
a, b, c = (100, 200, 300)

print(a)
print(b)
print(c)
```

Output

```
100
200
300
```

```
a, _, c = (100, 200, 300)
print(a)
print(c)
```

Output

```
100
300
```

```
a, *b = (1, 2, 3, 4, 5)
print(a)
print(b)
```

Output

```
1
[2, 3, 4, 5]
```

```
nested_tuple = (1, (2, 3), 4)
a, (b, c), d = nested_tuple

print(a)
print(b)
print(c)
print(d)
```

Output

```
1
2
3
4
```

Returning multiple values using tuples:
Python automatically packs the values into a tuple, which can then be unpacked into variables.

```
def fun():
    return "geeksforgeeks", 20
s, x = fun()
print(s)
print(x)
```

Output

```
geeksforgeeks
20
```

Dictionary:
It's a data structure that stores information as ***key-value*** pairs.
Key: unique and immutable elements (strings/numbers)

Value: elements of any data type; mutable/immutable

```
data = { "name": "Jake", "age": 22 }
print(data)
```

Output

```
{'name': 'Jake', 'age': 22}
```

It can also be created using the '***dict()***' function

```
a = {"x": 1, "y": 2}
print(a)

b = dict(name="Sam", age=20)
print(b)
```

Output

```
{'x': 1, 'y': 2}
{'name': 'Sam', 'age': 20}
```

Dictionary iteration:
It can be **traversed** using a '**for()**' loop to access its keys, values or both key-value pairs.

- Can be done using the '***keys()***', '***values()***' and '***items()***' methods.

```
d = {"a": 1, "b": 2}
for key in d:
    print(key)
```

Output

```
a
b
```

```
d = {"a": 1, "b": 2}
for value in d.values():
    print(value)
```

Output

```
1
2
```

```
d = {"a": 1, "b": 2}
for key, value in d.items():
    print(key, value)
```

Output

```
a 1
b 2
```

Nested dictionaries:
They are dictionaries that contain another dictionary as one of its values.

```
d = {
    "student": {
        "name": "Sam",
        "age": 20
    }
}

print(d["student"]["name"])
```

Output

```
Sam
```

Dictionary methods:
There are built-in methods to add, update, access and remove key-value pairs. They are as follows -

1. **clear()** —> removes all key-value pairs
2. **get()** —> returns the value associated with the specified key
3. **keys()** —> returns a view object containing all dictionary keys
4. **values()** —> returns a view object containing all dictionary values
5. **update()** —> updates the dictionary using another dictionary or iterable of key-value pairs
6. **pop()** —> removes the specified key and returns its value

Set and frozen set:

Set	Frozen set
Mutable	Immutable
Syntax: {1, 2, 3} or set(iterable)	Syntax: frozenset(iterable)
Supports modifying methods (add, remove, pop, update)	Only supports non-modifying methods (union, intersection)

```
s = set(["a", "b", "c"])
print("Normal Set:", s)

fs = frozenset(["e", "f", "g"])
print("Frozen Set:", fs)
```

Output

```
('Normal Set:', set(['a', 'c', 'b']))
('Frozen Set:', frozenset(['e', 'g', 'f']))
```

String indexing:

It provides access to individual characters in a string by their position/index.

- Can be done using both ***zero-based*** and ***negative*** indexing.

```
s = "ABCDEF"
print(s[0])
print(s[4])
```

Output

```
A
E
```

String slicing:

It is a way to extract a portion of a string by specifying the start and end indexes.

- Start index is included
  - End index is excluded

```
s = "ABCDEF"
print(s[1:4])
print(s[:3])
print(s[3:])
print(s[::-1])
```

Output

```
BCD
ABC
DEF
FEDCBA
```

String immutability:

Values of string can't be changed after creation. Any modification to a string creates a new one instead of altering the original one.

```
s = "aBCDEF"
s = "A" + s[1:]
print(s)
```

Output

```
ABCDEF
```

Common string methods:

1. **len()** —> returns the total number of characters in a string
2. **upper()** —> converts all characters to uppercase
3. **lower()** —> converts all characters to lowercase
4. **strip()** —> removes leading and trailing whitespaces from the string
5. **replace()** —> replaces all occurrences of a substring with another

Dictionary/set membership:

Used to test if a value/variable is in a sequence.
"***in***" and "***not in***" are the membership operators

```
x = 24
y = 20
my_list = [10, 20, 30, 40, 50]

if (x not in my_list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")

if (y in my_list):
    print("y is present in given list")
else:
    print("y is NOT present in given list")
```

Output

```
x is NOT present in given list
y is present in given list
```

Python object equality:

'==' and 'is' are comparison operators that serve different purposes.

- '**==**' checks whether two objects have equal values
  - '**is**' checks whether two variables reference the same object in memory

Two objects can contain identical values without sharing the same memory location.

```
a = [1,2,3]
b = [1,2,3]

print(a == b)
print(a is b)
```

Output

```
True
False
```

—> ‘a == b’ checks value equality and returns True.
—> ‘is’ checks memory identity and returnsFalse.

Counter:

It is a subclass from the ‘***Collections***’ module which is used to count the frequency of elements in lists, strings, dictionaries or tuples.

```
from collections import Counter
ctr1 = Counter([1, 2, 2, 3, 3, 3]) # From a list
ctr2 = Counter({1: 2, 2: 3, 3: 1}) # From a dictionary
ctr3 = Counter('hello') # From a string

print(ctr1)
print(ctr2)
print(ctr3)
```

Output

```
Counter({3: 3, 2: 2, 1: 1})
Counter({2: 3, 1: 2, 3: 1})
Counter({'l': 2, 'h': 1, 'e': 1, 'o': 1})
```

DefaultDict:

It is a subclass from the ‘***Collections***’ module. It automatically assigns a default

value to keys that do not exist.

```
from collections import defaultdict
d = defaultdict(list)

d['fruits'].append('apple')
d['vegetables'].append('carrot')
print(d)
print(d['juices'])
```

Output

```
defaultdict(<class 'list'>, {'fruits': ['apple'], 'vegetables': ['carrot']})
[]
```

Deque:

It stands for 'Double-ended Queue', which is a type of data structure that allows to add/remove elements from both ends efficiently.

- Allows fast insertion and deletion from both ends
- Works as both queue (FIFO) and stack (LIFO)

Operations -

1. **append()** —> adds an element to the right end of the queue.
2. **appendleft()** —> adds an element to the left end of the queue.
3. **extend()** —> adds multiple elements to the right end of the queue.
4. **extendleft()** —> adds multiple elements to the left end of the queue